Practical SIMD Vectorization Techniques for Intel® Xeon Phi™ Coprocessors

Xinmin Tian, Hideki Saito, Serguei V. Preis‡, Eric N. Garcia, Sergey S. Kozhukhov‡
Matt Masten, Aleksei G. Cherkasov‡ and Nikolay Panchenko‡

Mobile Computing and Compilers
Software and Service Group, Intel Corporation
Santa Clara, California, USA and ‡Novosibirsk, Russia
Email: {xinmin.tian, hideki.saito, serguei.v.preis, eric.n.garcia}@intel.com

***Abstract* — Intel® Xeon Phi™ coprocessor is based on the Intel® Many Integrated Core (Intel® MIC) architecture, which is an innovative new processor architecture that combines abundant thread parallelism with long SIMD vector units. Efficiently exploiting SIMD vector units is one of the most important aspects in achieving high performance of the application code running on Intel® Xeon Phi™ coprocessors. In this paper, we present several practical SIMD vectorization techniques such as less-than-full-vector loop vectorization, Intel® MIC specific alignment optimization, and small matrix transpose/multiplication 2-D vectorization implemented in the Intel® C/C++ and Fortran production compilers for Intel® Xeon Phi™ coprocessors. A set of workloads from several application domains is employed to conduct the performance study of our SIMD vectorization techniques. The performance results show that we achieved up to 12.5x performance gain on the Intel® Xeon Phi™ coprocessor.**

Keywords: SIMD vectorization, Intel® Xeon Phi™ coprocessor, Intel® MIC Architecture, compiler optimization

I. INTRODUCTION

The Intel® Xeon Phi™ coprocessor is based on the Intel® Many Integrated Core (Intel® MIC) architecture, which consists of many small, power efficient, in-order cores, each of which has a powerful 512-bit vector processing unit (SIMD unit) [9]. It is designed to serve the needs of applications that are highly parallel, make extensive use of SIMD vector operations, or are memory bandwidth bound. Hence, it is targeted for highly parallel, High Performance Computing (HPC) workloads [21] in a variety of fields such as computational physics, chemistry, biology, and financial services [19]. The Intel® Xeon Phi™ Coprocessor 5110P has the following key specifications:

- 60 cores, 240 threads (4 threads/core)
- 1.053GHz
- 1TeraFLOP double precision theoretical peak performance
- 8GB memory with 320GB/s bandwidth
- 512bit wide SIMD vector engine
- 32KB L1, 512KB L2 cache per core
- Fused Multiply-Add (FMA) support

One Teraflop theoretical peak performance is computed as follows: 1.053GHz × 60 cores × 8 double-precision elements in SIMD vector × 2 flops per FMA. As such, any compute bound applications trying to achieve high performance on Intel® Xeon Phi™ coprocessors need to exploit a high degree of parallelism and wide SIMD vectors. Using a 512-bit vector unit, 16 single precision (or 8 double precision) floating point (FP) operations can be performed as a single vector operation. With the help of the fused multiply-add (FMA) instruction, up to 32 FP operations can be performed at each core at each cycle. In comparison to the current 128-bit SSE and 256-bit AVX vector extensions, this new coprocessor can pack up to 8x and 4x the number of operations into a single instruction, respectively.

Wider SIMD vector units cannot be effectively utilized by simply extending the vectorizer for Intel® SSE and Intel AVX® architectures. Consider the following simple example. There exists a scalar loop that executes *N*-iterations. Using the vector length of *VL*, a vector loop would execute *floor(N/VL)* full vector iterations followed by *N mod VL* scalar remainder iterations. Unless *N* is sufficiently larger than *VL*, executing *N mod VL* scalar iterations can still be a significant portion of the vector execution of such a loop. In what follows, we will discuss two approaches in handling such "less than full vector" situations: The first technique is masked vectorization and the second technique is small matrix optimization and 2-dimentional (2-D) vectorization.

Furthermore, architectural/micro-architectural differences between Intel® Xeon Phi™ coprocessors and Intel® Xeon processors necessitates that new compiler techniques be developed. This paper focuses on three SIMD vectorization techniques and makes the following contributions.

- We propose an extended compiler scheme to vectorize short trip-count loops, peeling and remainder loops, that are classified as "less than full vector" cases, with a masking capability supported by the Intel® MIC architecture.
- We describe our specific data alignment strategies for achieving optimal performance through vectorization, as the Intel® MIC architecture is much more demanding on memory alignment than the Intel® AVX architecture [10].
- We describe our 2-Dimensional vectorization method which is beyond the conventional loop vectorization for small matrix transpose and multiplication operations by fully utilizing long SIMD vector units, swizzle, shuffle and masking support on the Intel® MIC architecture.

978-0-7695-4979-8/13 $26.00 © 2013 IEEE
DOI 10.1109/IPDPSW.2013.245

IEEE computer society

Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.

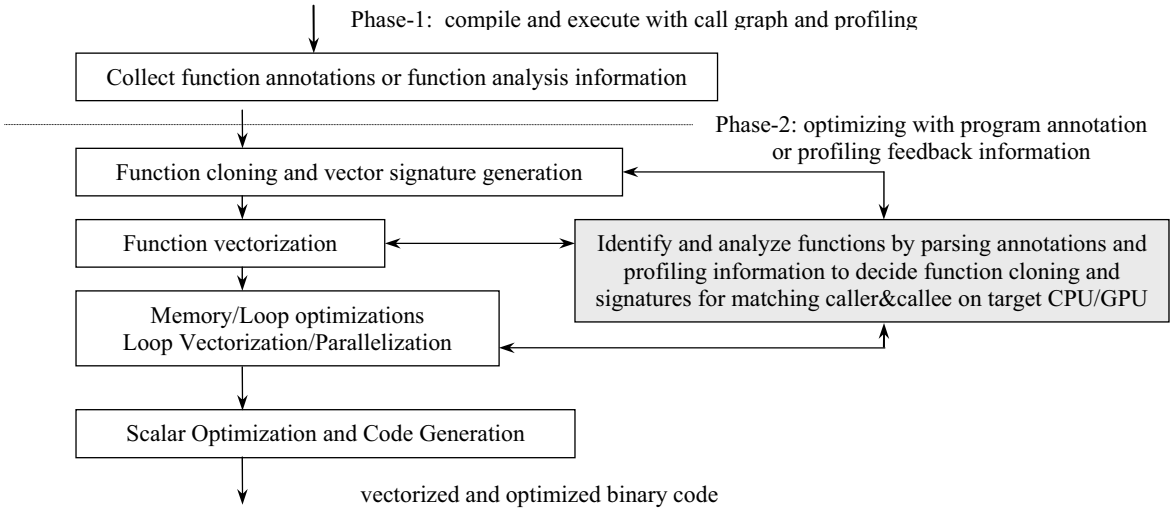


Figure 2.1. SIMD vector compilation infrastructure for function and loop vectorization

The rest of this paper is organized as follows: Section II provides a high-level overview of Intel® C/C++ and Fortran compilers. In Section III, the compiler details of "less than full vector" loop vectorization are described and discussed. Specific data alignment strategies for the Intel® Xeon Phi™ coprocessor and the schemes of performing data alignment optimization are discussed in Section IV. Section V presents the 2-D vectorization methods for small matrix transpose and multiplication. Section VI discusses related work. Section VII provides the performance results with a set of workloads and micro-benchmarks. Section VIII concludes the paper.

II. Compiler Architecture for Vectorization

This section describes the Intel® C/C++ and Fortran compiler support for the Intel® Xeon Phi™ coprocessor at a high level with respect to loop vectorization, and the translation and optimization of SIMD *vector* extensions [3, 4, 23]. The compiler translates serial C/C++ and Fortran code via automatic loop analysis or based on annotations using the *simd* pragma and *vector* attributes into SIMD instruction sequences. The compilation process is amenable to many optimizations such as loop parallelization, memory locality optimizations, classic loop transformations and optimizations, redundancy elimination, and dead code elimination before and after the loop/function vectorization. Figure 2.1 depicts the SIMD compilation infrastructure of the Intel® C/C++ and Fortran compilers for automatic loop vectorization and compiling *SIMD* pragma, *vector* function annotations and associated clauses. The framework consists of four major parts:

- Perform automatic loop analysis, identify and analyze programmer annotated functions and loops by parsing and collecting function and loop vector properties. In addition, our compiler framework can apply inter-procedural analysis and optimization with profiling and call-graph creation for automatic function vectorization.
- Generate vectorized function variants with properly constructed signatures via function cloning and vector signature generation.
- Vectorize *simd* for-loops that are identified by the compiler or annotated using SIMD extensions (#pragma *simd* can be used to vectorize outer loops), and cloned vector function bodies and all arguments by leveraging and extending our automatic loop vectorizer.
- Enable classical scalar, memory and loop optimizations, and parallelization effectively, before or after loop and function vectorization, for achieving good performance.

III. Less Than Full Vector Loop Vectorization with Masking

Intel® Xeon Phi™ coprocessor provides long (512-bit) SIMD vector hardware support for exploiting more vector-level parallelism. The long SIMD vector unit imposes the requirement of packing more scalar loop iterations into a single vector loop iteration, which also results in more iterations in the peeling loop, and/or in the remainder loop remaining non-vectorized, due to the fact that they don't constitute the full SIMD vector (or less than full vector) unit of Intel® MIC Architecture. For example, consider the short trip-count loop below.

```
float x, y[31];
for (k=0; k<31; k++) {
    x = x + fsqrt(y[k]);
}
```

When the loop is vectorized for Intel® SSE2 with vector length=4 (128-bit), the remainder loop will have 3 iterations. When the loop is vectorized for the Intel® MIC architecture with vector length=16 (512-bit), the remainder loop will have 15 iterations. In another situation, if the loop is unrolled by 16, then the remainder loop will have 15 iterations, leaving the

Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.

remaining 15 iterations in a scalar execution form. Thus, vectorizing the peeling and remainder loops (i.e. short trip-count loop in general) is very important for the Intel® MIC Architecture. This section describes how to apply vectorization, with masking support, to peeling and remainder loops (i.e. short trip-count loop) with special guarding masks to prevent the SIMD code from exceeding original loop and memory access boundaries. At a high-level, the following steps describe our vectorization scheme without vectorization of peeling and remainder loops.

- s0: Select alignment, vector length, and unroll factor
- s1: Generate alignment setup code
- s2: Compute the trip count of the peeling loop
- s3: Emit the scalar peeling loop
- s4: Generate the vector loop initialization code
- s5: Emit the main vector loop
- s6: Compute the trip count of the remainder loop
- s7: Emit the scalar remainder loop

Given the simple example below, the loop trip-count '*n*' and the pointer '*y*' (&y[0]) have a memory alignment that is unknown at compile time.

```
float foo(float *y, int n)
{  int k; float x = 10.0f;
   for (k = 0; k < n; k++) {
     x = x + fsqrt(y[k])
   }
   return x;
}
```

On the Intel® MIC Architecture the vector length is 512 bits, which requires 64-byte alignment for efficient memory accesses. To achieve 64-byte aligned memory loads/stores, we need to pack 16 float (32-bit) elements for each single vector iteration and generate a peeling loop. The following pseudo-code shows the vectorized loop based on the vectorization steps [s0, s1, …, s7] described above. The "less than full vector" loops, i.e. the peeling and remainder loops, are not vectorized.

Pseudo-code without vectorizing "less than full vector" loops

```
misalign = &y[0] & 63
peeledTripCount = (63 - misalign)/sizeof(float)
x = 10.0f;
do k0 = 0, peeledTripCount-1  // peeling loop
   x = x + fsqrt(y[k0])
enddo
x1_v512 = (m512)0
x2_v512 = (m512)0
mainTripCount = n - ((n - peeledTripCount) & 31)
do k1 = peeledTripCount, mainTripCount-1, 32
  x1_v512 = _mm512_add_ps(
                _mm512_fsqrt(y[k1:16]),x1_v512)
  x2_v512 = _mm512_add_ps(
                _mm512_fsqrt(y[k1+16:16]), x2_v512)
enddo
// perform vector add on two vector x1_v512 and x2_v512
x1_v512 = _mm512_add_ps(x1_v512, x2_512);

// perform horizontal add on all elements of x1_v512, and
// the add x for using its value in the remainder loop
x = x + _mm512_hadd_ps(x1_512)
do k2 = mainTripCount, n  // Remainder loop
   x = x + fsqrt(y[k2])
enddo
```

Note that we performed loop unrolling for the main vectorized loop, which allows the hardware to issue more instructions per cycle by hiding memory access latency and reducing branching. To enable the "less than full vector" (i.e. peeling loop, remainder loop, or short trip-count loop) vectorization, the loop vectorization scheme is extended as below.

- s0: Select alignment, vector length and unroll factor
- s1: Generate alignment setup code
- s2: Compute the trip count of peeling loop
  - create a vector of 16 elements with value <0, …, 15>
  - create a vector of 16 elements with value <peeledTripCount, …, peeledTripCount>
- s3: Emit the vectorized peeling loop with masking operations
- s4: Generate the main vector loop initialization code
- s5: Emit the main vector loop
- s6: Compute the trip count of the remainder loop
  - create a vector of 16 elements with the value <mainTripCount, …, mainTripCount+15>
  - create a vector of 16 elements with the value <origTripCount, …, origTripCount>
- s7: Emit the vectorized remainder loop with masking operations

The following pseudo-code shows the vectorized loops based on the extended vectorization schemes [s0, s1, …, s7] described as above.

Pseudo-code with vectorizing "less than full vector" loops using mask

```
misalign = &y[0] & 63
peeledTripCount = (63 - misalign) / sizeof(float)
x = 10.0f;
// create a vector: <0,1,2,…15>
k0_v512 = _mm512_series_pi(0, 1, 16)

// create vector: all 16 elements are peeledTripCount
peeledTripCount_v512 =
              _mm512_broadcast_pi32(peeledTripCount)
x1_v512 = (m512)0
x2_v512 = (m512)0
do k0 = 0, peeledTripCount-1, 16
  // generate mask for vectorizing peeling loop
  mask = _mm512_compare_pi32_mask_lt(k0_v512,
                                     peeledTriPCount_v512)
  x1_v512 = _mm512_add_ps_mask(
                _mm512_fsqrt(y[k0:16]), x1_v512, mask)
enddo

mainTripcount = n - ((n - peeledTripCount) & 31)
do k1 = peeledTripCount, mainTripCount-1, 32
  x1_v512 = _mm512_add_ps(
                _mm512_fsqrt(y[k1:16]),    x1_v512)
  x2_v512 = _mm512_add_ps(
                _mm512_fsqrt(y[k1+16:16]), x2_v512)
enddo
// create a vector: <mainTripCount,
                     mainTripCount+1 … mainTripCount+15>
k2_v512   = _mm512_series_pi(mainTripCount, 1, 16)

// create a vector: all 16 elements has the same value n
n_v512    = _mm512_broadcast_pi32(n)
step_v512 = _mm512_broadcast_pi32(16)

do k2 = mainTripCount, n, 16 // vectorized remainder loop
  mask    = _mm512_compare_pi32_mask_lt(k2_v512, n_v512)
  x1_v512 = _mm512_add_ps_mask(
                _mm512_fsqrt(y[k2:16]), x1_v512, mask)
  k2_v512 = _mm512_add_ps(k2_v512, step_v512)
enddo

x1_v512 = _mm512_add_ps(x1_v512, x2_512);
```


Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.

```
// perform horizontal add on 8 elements and final
// reduction sum to write the result back to x.
x = x + _mm512_hadd_ps(x1_512)
```

In the cases of short trip-count loop vectorization of peeling and remainder loops with runtime trip-count and alignment checking, loops are vectorized as efficiently as possible. These loops are vectorized with optimal vector lengths and an optimal amount of profitable unrolling regardless of a known loop trip count. This provides better utilization of SIMD vector hardware without sacrificing the performance of short loops. This scheme allows us to completely eliminate scalar execution of the loop in favor of masked SIMD vector code generation. Special properties of the mask are used to match unmasked code generation in most cases. E.g., masked scalar memory loads that could be unsafe under an empty mask are considered safe under a remainder mask since it is never empty.

Without adding the capability of short trip-count loop vectorization, the loops in the ConvolutionFFT2D benchmark with 7 iterations and double precision data type would end up as a fully scalar execution. Applying vectorization with masking to these short trip-count loops results in a ~2x to ~5x speedup for the 7-iteration short trip-count (or less than full vector) loops in the ConvolutionFFT2D benchmarks on the Intel® MIC Architecture.

IV. Alignment Strategy and Optimization

The Intel® Xeon Phi™ coprocessor is much more sensitive to data alignment than the Intel® Xeon E5 processor, so developing an Intel® MIC oriented alignment strategy and optimization schemes is one of the key aspects for achieving optimal performance.

- Similar to Intel® SSE4.2, the SIMD load+op instructions require vector size alignment, which is 64 byte alignment for the Intel® MIC architecture. However, simple load/store instructions require the alignment information to be known at compile time on the Intel® Xeon Phi™ coprocessor.
- Different from prior Intel® SIMD extensions, all SIMD load/store instructions including gather/scatter require at least element size alignment. Misaligned elements will cause a fault. This necessitates the Intel® MIC architecture ABI [15] to require that all memory accesses be element-wise aligned.
- There are no special unaligned load/store instructions in the Intel® Initial Many Core Instruction (Intel® IMCI) set. This is overcome by using unpacking loads and packing stores that are capable of dealing with unaligned (element-aligned) memory locations. Due to their unpacking and packing nature, these instructions cannot be directly used for masked loads/stores, except under special circumstances.
- The faulting nature of masked memory access instructions in Intel® IMCI adds extra complexity to those instructions addressing data outside paged memory, and may fail even if actual data access is masked out. The exceptions are gather/scatter instructions.

Therefore, the compiler aggressively performs data alignment optimizations using traditional techniques such as alignment peeling and alignment multi-versioning.

Alignment peeling implies the creation of a pre-loop that executes several iterations on unaligned data in order to reach an aligned memory address. As a result, most of these iterations are executed using aligned SIMD operations. The pre-loop can be vectorized with masking as described in Section II. Unfortunately, this scheme works only for one set of co-aligned memory addresses, and the others are assumed to be unaligned. In addition, our multi-versioning optimization can be applied to the second set of co-aligned locations by examining them dynamically. Aligned or unaligned operations are used based on the results of the examination.

For unmasked unaligned (element-aligned) vector loads and stores, the compiler uses unpacking/packing load and store instructions. They are safe in this scenario and perform much better than gather/scatter instructions. If the compiler cannot prove the safety of the entire address range of a particular memory access, it inserts a zero-mask check in order to avoid a memory fault. All instructions with the same mask are emitted under a single check to avoid execution under the empty mask, and to eliminate multiple checks of the same condition.

Unpacking and packing instructions may cause fault when they are used with a mask, as they may address masked-out invalid memory. On-the-fly data conversion may cause fault even without masking. Thus, for unaligned masked and/or converting loads/stores, the compiler uses gather/scatter instructions instead for safety, even though this degrades performance. Memory faults would never happen if each memory access had at least one vector (64 bytes) of memory paged after its initial address. This can be achieved by padding each data section in the program and each dynamically allocated object with 64 bytes. For developers who are willing to do the padding to achieve optimal performance from masked code, the compiler knob –opt-assume-safe-padding was introduced. Under this knob, unaligned masked and/or converting load/store operations are emitted as unpacking loads/packing stores:

- In unmasked converting cases, as well as for cases with peel/remainder masks, the compiler emits loads/stores directly. The mask in this case will work since it is dense.
- For an arbitrary masking scenario, an unmasked load unpack instruction is used, which is safe due to the padding assumption, followed by a masked move (blend). The ‘non-empty-mask’ check guarantees that the 64-byte padding is always enough for safety, that is,

Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.

at least one item within the vector is to be loaded. Thus, the tail end of the memory access is within 64 bytes from meaningful data.

The safe-padding optimization has provided notable improvements on a number of benchmarks, e.g. 10% gain on BlackScholes and selected Molecular Dynamics kernels.

V. SMALL MATRIX OPERATIONS 2-D VECTORIZATION

Frequently seen in HPC workloads, operations on small matrices are a growing, profitable set of calculations for vectorization on Intel® Xeon Phi™ coprocessors. With the wider SIMD unit support, the Intel® C/C++ and Fortran compilers are enhanced to vectorize common operations on small matrices along 2 dimensions. Small matrices are matrices whose data can reside entirely in one or two 512-bit SIMD registers. Consider the example Fortran loop nest with 32-bit float (or real) type below:

```
real, dimension(4,4) :: A, B, C
real sum
integer  j, l, i
do j = 1, 4
  do l = 1, 4
    sum = 0.0
    do i = 1, 4
      sum = sum + A(i,l) * B(i,j)
    enddo
    C(l,j) = sum
  enddo
enddo
```

Example 5.1: Small Matrix Multiplication Summation

With non-unit stride references present in the inner loop of Example 5.1, the conventional inner loop vectorization will not provide the most efficient vectorization of the loop nest. The outer loop vectorization faces similar issues. The Intel® C/C++ and Fortran compiler employs the wider SIMD vector unit of the Intel® MIC architecture and vectorizes this example loop across all three loop nesting levels, named as 2-Dimentional (2-D) vectorization on small matrices.

The vectorization approach is detailed below with vector intrinsic pseudo-code. For visualization, Figures 5.1−5.13 depict a snapshot of the various vector unit contents after each corresponding instruction. The tables in each figure represent a vector unit, whose name is in the leftmost column and its contents in the rightmost four columns. Of the rightmost four columns, the lowest addressed element is in the top left corner and each consecutive element follows a row-major addressing order.

First, array data is loaded into a vector unit. With a wider SIMD vector unit, the compiler is able to load the entire A and B matrix each into a single vector unit.

a) Matrix A and B are loaded into two SIMD registers.

```
// Load A matrix from memory into vector register.
A_v512 = <A[1][1], A[1][2], … … , A[4][3], A[4][4]>
```

A_v512	A[1][1]	A[1][2]	A[1][3]	A[1][4]
	A[2][1]	A[2][2]	A[2][3]	A[2][4]
	A[3][1]	A[3][2]	A[3][3]	A[3][4]
	A[4][1]	A[4][2]	A[4][3]	A[4][4]

Figure 5.1: Contents of vector register A_v512 after load.

```
// Load B matrix from memory into vector register.
B_v512 = <B[1][1], B[1][2], … … , B[4][3], B[4][4]>
```

B_v512	B[1][1]	B[1][2]	B[1][3]	B[1][4]
	B[2][1]	B[2][2]	B[2][3]	B[2][4]
	B[3][1]	B[3][2]	B[3][3]	B[3][4]
	B[4][1]	B[4][2]	B[4][3]	B[4][4]

Figure 5.2: Contents of vector register B_v512 after load.

Next, the compiler optimizes the multiplication operation between matrix A and matrix B, through a series of data layout transformations and vector multiplication and addition operations. The compiler identifies a matrix multiplication in this loop and permutes the elements in matrix A and matrix B setting up simple vector multiplications and additions.

b) We can simplify the multiplication needed through a transposition of the elements of A, followed a multiply and add of each row B and with each row of transposed A. We start by transposing the elements of A.

```
// First, create a vector unit of zeros.
A'_v512 = _mm512_setzero()
```

A'_v512	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

Figure 5.3: A'_v512 after zero initialization.

For the transpose operation, we use a set of new Intel® MIC _mm512_mask_shuf128x32() intrinsic calls. Similarly in classic architectures, this shuffle intrinsic is bound by four 128-bit "lanes" in each vector register. Thus, this intrinsic contains arguments for permute patterns for each of the four 128-bit lanes, as well as a permute pattern for each of the four 32 bit boundaries within each of those lanes. The arguments are as follows:

```
_m512 res = _mm512_mask_shuf128x32(_m512 v1,
                (I16) vmask, _m512 v2, (SI32)perm128, (SI32)perm32)
```

- res : result vector unit.
- v1: blend-to vector unit. The values in this vector unit will be blended with the shuffled elements of the v2, according to the write mask.
- vmask : write mask. The write mask is a bit vector specifying which elements to overwrite in v1 with the shuffle elements of v2.
- v2: incoming data vector unit. This vector unit holds the elements which are to be shuffled.
- perm128: 128-bit lane permutation. This value specifies the permutation order of the vector unit's 128-bit lanes.
- perm32: element-wise permutation. This value specifies the permutation order of the each of the four 32 bit boundaries within each 128-bit lane.

```
// Begin transpose operation by shuffling elements into
```


Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.

```
// desired order. Shuffle used to insert matrix diagonal
// into transpose result vector unit.
A'_v512 = _mm512_mask_shuf128x32(A'_v512, 0x8421,
                  A_v512, _MM_PERM_DCBA, _MM_PERM_DCBA)
```

A'_v512	A[1][1]	0	0	0
	0	A[2][2]	0	0
	0	0	A[3][3]	0
	0	0	0	A[4][4]

Figure 5.4: Vector register contents after first shuffle.

```
// Shuffle the next four elements and blend-in with the
// elements written from previous shuffle.
A'_v512 = _mm512_mask_shuf128x32(A'_v512, 0x4218,
                  A_v512, _MM_PERM_CBAD, _MM_PERM_ADCB)
```

A'_v512	A[1][1]	0	0	A[4][1]
	A[1][2]	A[2][2]	0	0
	0	A[2][3]	A[3][3]	0
	0	0	A[3][4]	A[4][4]

Figure 5.5: Vector register contents after second shuffle.

```
// Shuffle the next four elements and blend-in with the
// elements written from previous shuffle.
A'_v512 = _mm512_mask_shuf128x32(A'_v512, 0x2184,
                  A_v512, _MM_PERM_BADC, _MM_PERM_BADC)
```

A'_v512	A[1][1]	0	A[3][1]	A[4][1]
	A[1][2]	A[2][2]	0	A[4][2]
	A[1][3]	A[2][3]	A[3][3]	0
	0	A[2][4]	A[3][4]	A[4][4]

Figure 5.6: Vector register contents after third shuffle.

```
// Shuffle the final four elements and blend-in with the
// elements written from previous shuffle to obtain the
// complete transpose.
A'_v512 = _mm512_mask_shuf128x32(A'_v512, 0x1842,
                  A_v512, _MM_PERM_ADCB, _MM_PERM_CBAD)
```

A'_v512	A[1][1]	A[2][1]	A[3][1]	A[4][1]
	A[1][2]	A[2][2]	A[3][2]	A[4][2]
	A[1][3]	A[2][3]	A[3][3]	A[4][3]
	A[1][4]	A[2][4]	A[3][4]	A[4][4]

Figure 5.7: Vector register contents after the final shuffle.

After the elements of matrix A have been permuted through transposition, each element of A and B is now in the correct position within each vector unit for a vector product, resulting in the same behavior as the dot product of rows and columns.

c) Next, we perform the multiplication of each row of the transposed A with each row of B, maintaining a sum of the products from row to row.

```
// Load the first row of A'_v512 and broadcast that row to
// each of the remaining three rows
t1_v512 = _mm512_extload_ps(A'_v512[0:4],
                _MM_FULLUPC_NONE, _MM_BROADCAST_4X16, 0)
```

t1_v512	A[1][1]	A[2][1]	A[3][1]	A[4][1]
	A[1][1]	A[2][1]	A[3][1]	A[4][1]
	A[1][1]	A[2][1]	A[3][1]	A[4][1]
	A[1][1]	A[2][1]	A[3][1]	A[4][1]

Figure 4.8: Vector register contents after load with broadcast

Another useful intrinsic used in this optimization is the Intel® MIC _mm512_swizzle_ps() intrinsic. This intrinsic is similar to that of the shuffle above except it only permutes each 128-bit lane and not each of the 32 boundaries within those lanes. The arguments are as follows:

_m512 res = _mm512_swizzle_ps(_mm512 v1, SI32 perm)

- res : result vector unit.
- v1 : incoming data vector unit to be permuted.
- perm: permutation pattern for each 128-bit lane.

```
// Load the first row of B_v512 and broadcast that row to
// each of the remaining three rows
t2_v512 = _mm512_swizzle_ps(B_v512,_MM_SWIZ_REG_AAAA)
```

t2_v512	B[1][1]	B[1][2]	B[1][3]	B[1][4]
	B[1][1]	B[1][2]	B[1][3]	B[1][4]
	B[1][1]	B[1][2]	B[1][3]	B[1][4]
	B[1][1]	B[1][2]	B[1][3]	B[1][4]

Figure 5.9: Vector register contents illustrating swizzle.

```
// Multiply each element of t1_v512 with each element of
// t2_v512 and store result in C_v512
C_v512 = _mm512_mul_ps(t1_v512, t2_v512)
```

C_v512	t1_v512 * t2_v512

Figure 5.10: C_v512 vector unit contains element-wise product of t1_v512 and t2_v512

```
// Load the second row of A'_v512 and broadcast that row
// to each of the remaining three rows
t1_v512 = _mm512_extload_ps(A'_v512[4:8],
                  _MM_FULLUPC_NONE, _MM_BROADCAST_4X16, 0)

// Load the second row of B_v512 and broadcast that row to
// each of the remaining three rows
t2_v512 = _mm512_swizzle_ps(B_v512,_MM_SWIZ_REG_BBBB)
```

Each subsequent multiplication must be accumulated for each row. These multiplies and adds are the corresponding dot product of rows and columns found in matrix multiplication, but because of the earlier transpose, no further permuting is required.

```
// Add the existing values of C_v512 with the product of
// t1_v512 and t2_v512 and store result in C_v512
C_v512 = _mm512_madd213_ps(t2_v512, t1_v512, C_v512)

// Load the third row of A'_v512 and broadcast that row to
// each of the remaining three rows
t1_v512 = _mm512_extload_ps(A'_v512[8:12],
                  _MM_FULLUPC_NONE, _MM_BROADCAST_4X16, 0)

// Load the third row of B_v512 and broadcast that row to
// each of the remaining three rows
t2_v512 = _m512_swizzle_ps(B_v512, _MM_SWIZ_REG_CCCC)

// Add the existing values of C_v512 with the product of
// t1_v512 and t2_v512 and store result in C_v512
C_v512 = _mm512_madd213_ps(t2_v512, t1_v512,C_v512)

// Load the fourth row of A'_v512 and broadcast that row
// to each of the remaining three rows
t1_v512 = _mm512_extload_ps(A'_v512[12:16],
                  _MM_FULLUPC_NONE, _MM_BROADCAST_4X16, 0)
```

t1_v512	A[1][4]	A[2][4]	A[3][4]	A[4][4]
	A[1][4]	A[2][4]	A[3][4]	A[4][4]
	A[1][4]	A[2][4]	A[3][4]	A[4][4]
	A[1][4]	A[2][4]	A[3][4]	A[4][4]

Figure 5.11: t1_v512 vector register contents illustrating final load with broadcast.

```
// Load the fourth row of B_v512 and broadcast that row to
// each of the remaining three rows
t2_v512 = _mm512_swizzle_ps(B_v512, _MM_SWIZ_REG_DDDD)
```


Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.

t2_v512	B[4][1]	B[4][2]	B[4][3]	B[4][4]
	B[4][1]	B[4][2]	B[4][3]	B[4][4]
	B[4][1]	B[4][2]	B[4][3]	B[4][4]
	B[4][1]	B[4][2]	B[4][3]	B[4][4]

Figure 5.12: t2_v512 vector register contents illustrating final swizzle.

```
// Add the existing values of C_v512 with the product of
// t1_v512 and t2_v512 and store result in C_v512
C_v512 = _mm512_madd213_ps(t2_v512, t1_v512, C_v512)
```

C_v512	t2_v512 * t1_v512 + C_v512

Figure 5.13: Final C_v512 vector unit contains sum of existing values of C_v512 and element-wise products t2_v512 and t_v512

After the simplified matrix multiplication, the loop further requires that results be stored in the C matrix. With all elements correctly computed and residing in vector unit only one store operation is generated.

d) Finally, the result vector unit of values is stored to the C array.

```
// The elements of vector register C_v512 are then stored
// to memory at &C[1][1]
      <C[1][1], C[1][2], … C[4][3], C[4][4]> = C_v512
```

The 512-bit long SIMD vector unit of the Intel® MIC architecture supports consumption of both matrix dimensions for 2-D vectorization, fitting an entire small matrix (4x4 float type) into one 512-bit SIMD vector register. This enables more efficient flexible vectorization and optimizations for small matrix operations. For example, the scalar version of single precision 4x4 matrix multiply computation naively executes 128 memory loads, 64 multiplies, 64 additions, and 16 memory stores. The small matrix 2-D vectorization reduces instructions to 2 vector loads from memory, 4 multiplies, 4 shuffles, 4 swizzles, 3 additions, and 1 vector store to memory for a reduction of approximately 15x in number of instructions.

VI. Performance Evaluation

This section presents the performance results measured on an Intel® Xeon Phi™ coprocessor system using a set of workloads and micro-benchmarks.

A. Workloads

We have selected a set of workloads to demonstrate the performance benefits and importance of SIMD vectorization on the Intel® MIC Architecture. These workloads exhibit a wide range of application behavior that can be found in areas such as high performance computing, financial services, databases, image processing, searching, and other domains. These workloads include:

NBody*:* NBody computations are used in many scientific applications such as astrophysics [1] and statistical learning algorithms [8]. The main computation involves two loops that iterate over the bodies and computes a pair-wise interaction between them.

2D 5x5 Convolution*:* Convolution is a common image filtering computation used to apply effects such as blur and sharpen. For a given 2D image and a 5x5 spatial filter containing weights, this convolution computes the weighted sum for the neighborhood of the 5x5 set of pixels.

Back Projection*:* Back projection is commonly used for performing cone-beam image reconstruction of CT projection values [11]. The input consists of a set of 2D images that are "back-projected" onto a 3D volume in order to construct a 3D grid of density values.

Radar (1D Convolution)*:* The 1D convolution is widely used in applications such as radar tracking, graphics, image processing, etc.

Tree Search*:* In-memory tree structured index search is a commonly used operation in database applications. This benchmark consists of multiple parallel searches over a tree with different queries, where the path through the tree is determined based on the comparison of results of the query and node value at each tree level.

B. System Configuration

The detailed information on the configuration of the Intel® Xeon Phi™ Coprocessor used for the performance study and for evaluating the effectiveness of SIMD vectorization techniques is provided in Table 6.1.

System Parameters	Intel® Xeon Phi™ Processor
Chips	1
Cores/Threads	61 and 244
Frequency	1 GHz
Data caches	32 KB L1, 512 KB L2 per core
Power Budget	300 W
Memory Capacity	7936 MB
Memory Technology	GDDR5
Memory Speed	2.75 (GHz) (5.5 GT/s)
Memory Channels	16
Memory Data Width	32 bits
Peak Memory Bandwidth	352 GB/s
SIMD vector length	512 bits

TABLE 6.1 TARGET SYSTEM CONFIGURATION

C. Performance Results

All benchmarks were compiled as native executables using the Intel® 13.0 product compilers and run on the Intel® Xeon Phi™ coprocessor system specified in Table 6.1. To demonstrate the performance gains obtained through the SIMD vectorization, two versions of the binaries were generated for each workload. The baseline version was compiled with OpenMP parallelization only (-mmic -openmp -novec), the vectorized version is compiled with vectorization (default ON) and OpenMP parallelization (-mmic –openmp).

The performance scaling is derived from the OpenMP – only execution and OpenMP with 512-bit SIMD vector execution on the Intel® Xeon Phi™ coprocessor system we described at beginning of this section. That is, when the

Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.

workload contains 32-bit single-precision computations, 16-way vectorization may be achieved. When the workload contains 64-bit double-precision computations, 8-way vectorization is achieved.

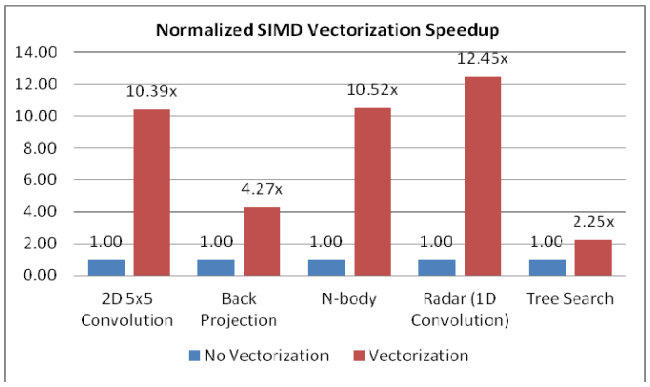


FIGURE 6.1. PERFORMANCE RESULTS OF WORKLOADS

Figure 6.1 shows the normalized SIMD performance speedups of five workloads. The generated SIMD code of these workloads achieved SIMD speedups ranging from 2.25x to 12.45x. Besides those classical HPC applications with regular array accesses and computations, the workload with a large amount of branching codes, such as Tree Search used in database applications, achieves 2.25x speedup as well with SIMD vectorzation based on the masking support in the Intel® MIC architecture.

D. Impact of Less-Than-Full-Vector Loop Vectorization

To examine the impact of the less-than-full-vector loop vectorization, a simple micro-benchmark was written with three small kernel functions: `intAdd`, `floatAdd`, and `doubleAdd`. Each of them has a short trip-count loop that takes 3 arrays, a, b, c of size 31 and does an element-wise addition with respect to `int`, `float` and `double` data types. The vector length is 16 iterations for loops in the `intAdd` and `floatAdd` kernels, and 8 iterations for the loop in the `doubleAdd` kernel function. This experimental setup ensures the `intAdd` and `floatAdd` loops contain a 15-iteration remainder loops, and the `doubleAdd` loop contains a 7-iteration remainder loop which can be vectorized with the "**less-than-full-vector**" loop SIMD vectorization technique using masking support described in the Section II.

Figure 6.2 shows performance gains from vectorization without "**less-than-full-vector**" loop vectorization and with "**less-than-full-vector**" loop vectorization for three short trip-count loops in the `intAdd`, `floatAdd` and `doubleAdd` kernel functions. The generated SIMD code of these loops achieves a speedup ranging from 2.89x to 3.32x without "**less-than-full-vector**" loop vectorization. With "**less-than-full-vector**" loop vectorization, the performance speedup is improved significantly, and ranges from 3.28x to 7.68x. Note that in this measurement, all data are 64-byte aligned, there are no peeling loops generated, and the aligned memory load/store instructions such as `vmovaps` and `vmovapd` [9] are generated to achieve optimal performance. The next sub-section shows the data alignment impact on the Intel® MIC architecture.

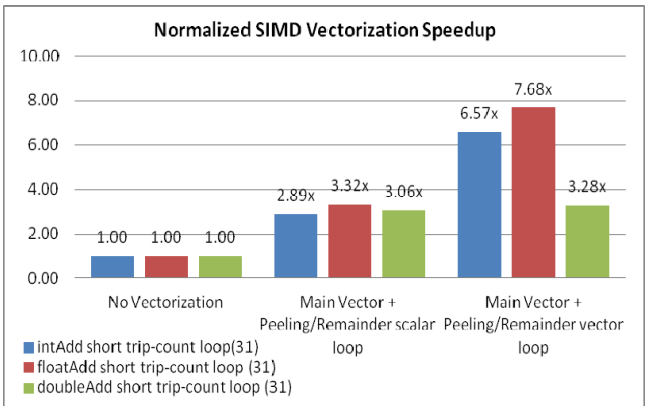


FIGURE 6.2. PERFORMANCE GAIN WITH "LESS-THAN-FULL-VECTOR" LOOP VECTORIZATION

E. Impact of Data Alignment

These kernel loops used in Sub-section *D* are re-used for this measurement. In this study, the difference is that we don't provide alignment information of the array `a`, `b`, and `c`. Without alignment information, given these loops are short trip-count loops with constant trip count, the compiler generates SIMD instructions:

- `vloadunpackld` and `vloadunpackhd` to load data from unaligned memory locations, `vpackstoreld` and `vpackstorehd` [9] to store data to unaligned memory locations for the vectorized main loop.
- `vgatherdps` and `vscatterdps` instructions [9] to load and store for the vectorized remainder loop with write-mask.

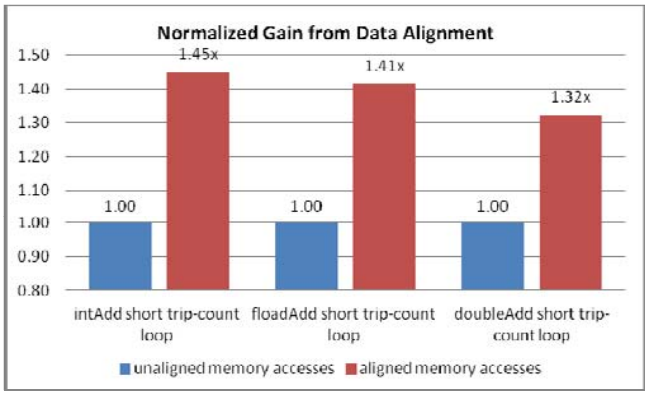


FIGURE 6.3. PERFORMANCE GAIN WITH DATA ALIGNMENT

As shown in Figure 6.3, with data alignment information, the performance of SIMD execution is 1.45x, 1.41x and 1.32x better than unaligned cases w.r.t `int`, `float`, and `double` types of three kernel functions. The alignment

Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.

optimization described in Section III is critical to achieving optimal performance on Intel® MIC architecture.

F. *Impact of Small Matrix 2-D Vectorization*

Small matrix operations such as addition and multiply have served as important parts of many HPC applications. A number of classic compiler optimizations such as loop complete unrolling, partial redundancy elimination (PRE), scalar replacement, and partial summation have been developed to achieve optimal vector execution performance. The conventional inner or outer loop vectorization for 3-level loop nests of 4x4 matrix operations is not performing well on Intel® Xeon Phi™ coprocessor due to:

- Less effective use of 512-bit long SIMD unit, e.g. for 32-bit float data type, when either inner loop or outer loop is vectorized. In this case 4-way vectorization is used instead of 16-way vectorization.
- Side-effects on classic optimizations, e.g. the partial redundancy elimination, partial summation, operator strength reduction, etc. when the loop is vectorized.

As shown in Figure 6.4, the convention loop vectorization on small matrix (4x4) operations does cause performance degradation. For both cases of single-precision and double-precision matrix (4x4) multiplications, the performance degradation is ~50% when comparing against cases without vectorization, which are used as the baseline performance.

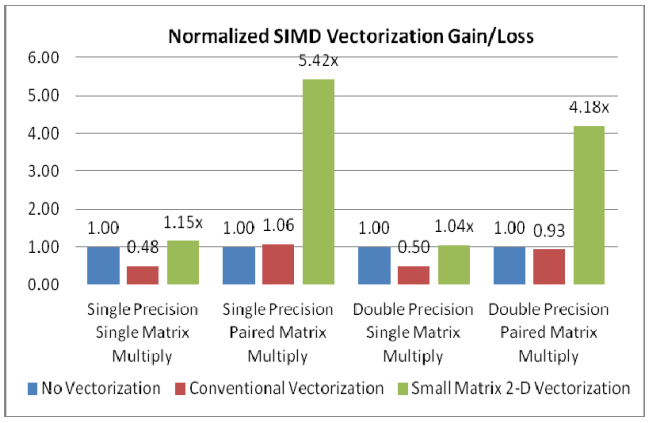


FIGURE 6.4. PERFORMANCE GAIN/LOSS WITH SIMD VECTORIZATION

In the case of the paired matrix multiplication, there are two matrix (4x4) multiplications are done in a single loop nest, and `matrixB` is transposed for computing `sumy`.

```
do j = 1, 4
  do k = 1, 4
    sumx = 0.0
    sumy = 0.0
    do i = 1, 4
      sumx = sumx + matrixA(i,k) * matrixB(i,j)
      sumy = sumy + matrixA(i,k) * matrixB(j,i)
    enddo
    matrixC(k,j) = sumx
    matirxD(j,k) = sumy
  enddo
enddo
```

The classical loop optimizations are not as effective as for the single matrix multiplication case due to the transpose operation of `matrixB` and paired matrix multiplications in the loop. Thus, the performance achieved with classical loop optimization is on-par with applying conventional loop vectorization, and no notable performance difference is observed as shown in Figure 6.4. Promisingly, applying the small matrix 2-D vectorization we proposed in Section IV, we achieved a performance speedup 1.15x / 1.04x for single matrix (4x4 float/double type) multiplication, and a speedup 5.42x / 4.18x for paired matrix (4x4 float/double type) transpose and multiplication, which demonstrates the effectiveness of small matrix 2-D vectorization using long SIMD vector unit supported by Intel® Xeon Phi™ coprocessor.

VII. RELATED WORK

The compiler vectorization technology [2] had been one of the key loop transformations for traditional vector machine decades ago. However, the recent proliferation of modern SIMD architectures [9, 10] poses new constraints such as data alignment, masking for control flow, non-unit stride access to memory, and the fixed-length nature of SIMD vectors, that shall demand more advanced vectorization technologies and vectorization friendly programming language extensions [23].

In the past three plus decades, the rich body of SIMD vectorization capabilities has been incorporated in a number of industry and research compilers [2, 3, 4, 7, 12, 14, 24]. These include works based on ICC (the Intel compiler) [3, 4], XLC (the IBM compiler) [7, 24], VAST [6], GCC [16, 17], and the SUIF compiler [5]. However, there are many unknown program factors such as loop trip count, memory access stride and patterns, alignment and control flow complexity at compile-time, that pose challenges to the modern optimizing compiler's ability to apply advanced and practical vectorization techniques, and fulfill the semantic gap between application programs and the modern processors such as Intel® Xeon Phi™ coprocessor for harnessing its computational power.

Compared to the conventional loop vectorization[2,3,5], the "less-than-full-vector" vectorization technique brings extra performance benefits for those vectorizable short trip-count loops, especially when the processor provides the long SIMD unit masking capability like the Intel® Xeon Phi™ coprocessor. Our alignment optimizations are built on top of existing dynamic alignment optimizations as presented in [3, 4]. However, the alignment strategy described in this paper is designed to satisfy the requirement of Intel® MIC architecture with optimal SIMD instruction selection and mask utilization for safe and optimal performance. Beyond traditional single-level loop vectorization [2, 3, 16, 17, 22, 24] , the small matrix operation 2-D vectorization increases vector-parallelism and

Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.

improves the utilization efficiency of the long SIMD vector unit, swizzle, shuffle, broadcast and mask support in Intel® MIC architecture significantly.

In addition, Programming language extensions such as OpenMP* SIMD extensions [13, 18] and Cilk™ Plus [19, 23] function vectorization and loop vectorization through the compiler has been paving the way to enable more effective vector–level parallelism [13, 23] in both C/C++ and Fortran programming languages. To support these SIMD vector programming models on the Intel® Xeon Phi™ coprocessor effectively, the practical vectorization techniques described in this paper are essential for achieving optimal performance and ensuring SIMD code execution safety on an Intel® Xeon Phi™ coprocessor system.

VIII. CONCLUSIONS

Driven by the increasing prevalence of SIMD architectures in the Intel® Xeon Phi coprocessor, we proposed and implemented new vectorization techniques to explore the effective use of its long SIMD units. This paper presented several practical SIMD vectorization techniques such as less-than-full-vector loop vectorization, Intel® MIC specific data alignment optimizations, and small matrix operations 2-D vectorization for the Intel® Xeon Phi™ coprocessor. A set of workloads from several domains was employed to evaluate the benefits of our SIMD vectorization techniques. The results show that we achieved up to 12.5x performance gain on Intel® Xeon Phi™ coprocessor.

Intel® C/C++ and Fortran compilers are highly enhanced for programmers to harness the computational power of Intel® Xeon Phi™ coprocessors for accelerating highly parallel applications found in chemistry, visual computing, computational physics, biology, financial services, pixel, multimedia, graphics and HPC applications by effectively exploiting the use of the Intel® MIC architecture SIMD vector unit beyond traditional loop SIMD vectorization.

REFERENCES

[1] Aarseth,S.J., Gravitational n-body Simulations Tools and Algorithms. *2003.*

[2] Allen, R. and K. Kennedy, K., "Automatic translation of FORTRAN programs to vector form," *ACM Trans. Program. Lang. Syst.*, vol. 9, no. 4, pp. 491–542, 1987.

[3] Bik, A. J., Girkar, M., Grey, P. M., and Tian, X., .Automatic Intra-Register Vectorization for the Intel Architecture. *International Journal of Parallel Programming*, (2):65-98, April 2002.

[4] Bik, A. J. C, Kreitzer, D., Tian, X.: A Case Study on Compiler Optimizations for the Intel® Core™ 2 Duo Processor. International Journal of Parallel Programming 36 (6): 571-591, 2008

[5] Cheong, G., and Lam, M. S., An Optimizer for Multimedia Instruction Sets, In Second SUIF Compiler Workshop, August 1997

[6] Crescent Bay Software. VAST-F/AltiVec: Automatic Fortran Vectorizer for PowerPC Vector Unit. http://www.psrv.com/vast altivec.html, 2004.

[7] Eichenberger, A., O'Brien, K., , Wu, P., Chen, T., el.al,. Optimizing Compiler for the CELL Processor. In *Proceedings of the 14th international Conference on Parallel Architectures and Compilation Techniques*, 2005.

[8] Gray, A.G. and A. W. Moore, A. W., 'N-Body' Problems in Statistical Learning. In *NIPS*, pages 521-527, 2000..

[9] Intel Corporation, "Intel® Xeon Phi™ Coprocessor System Software Developers Guide," November 2012, http://software.intel.com/en-us/mic-developer.

[10] Intel Corporation. Intel® Advanced Vector Extensions Programming Reference, Document number 319433-011, June 2011.

[11] Kachelrieb, M., Knaup, M., Bockenbach, O., Hyperfast perspective cone-beam backprojection. *IEEE Nuclear Sci.*, pp. 1679-1683, 2006.

[12] Karrenberg, R., Hack, S.: Whole-Function Vectorization. In: Proc. of the 9th Intl. Ann. IEEE/ACM Symp. on Code Generation and Optimization,, Charmonix, France, pp. 141–150 April 2011

[13] Klemm, M. Duran, A., Tian, X., Saito, H., Caballero, D., Martorell, X.: Extending OpenMP* with Vector Constructs for Modern Multicore SIMD Architectures. IWOMP 2012: pp.59-72.

[14] Larsen, S. and Amarasinghe, S., Exploiting Superword Level Parallelism with Multimedia Instruction Sets. In *Proceedings of the SIGPLAN Conference on Programming Language Design and Implementation*, pages 145{156, June 2000.

[15] Lu, H.J., Garkar, M., Matz, M., Hubicka, J., Jaeger, A., Mitchell, M. "System V Application Binary Interface K1OM Architecture Processor Supplement", Version 1.0, http://software.intel.com/en-us/forums/topic/278102, April 26, 2012,

[16] Nuzman, D., Zaks, A.: Outer-loop Vectorization: Revisited for Short SIMD Architectures. In: Proc. of the 17th Intl. Conf. on Parallel Architectures and Compilation Techniques, Toronto, ON, Canada, pp. 2–11, October 2008

[17] Nuzman, D., Henderson, R.: Multi-platform Auto-vectorization. In: Proc. of the 4th Ann. IEEE/ACM Intl. Symp. on Code Generation and Optimization, New York, pp. 281–294, March 2006

[18] OpenMP Architecture Review Board, "OpenMP Application Program Interface," Version 4.0 (Release Candidate RC1), Novmeber 2012, http://www.openmp.org

[19] Reinders, J., "An Overview of Programming for Intel® Xeon processor and Intel® Xeon Phi™ Coprocessor," 2012, http://software.intel.com/enus/mic-developer

[20] Ren, G., Wu, P. and Padua, D., A preliminary study on the Vectorization of multimedia applications for multimedia extensions. *In 16th International Workshop of Languages and Compilers for Parallel Computing*, October 2003.

[21] Satish, N., Kim, C., J. Chhugani, J., H. Saito, H., Krishnaiyer, R., M. Smelyanskiy, M., Girkar, M., and Dubey, P."Can traditional programming bridge the ninja performance gap for parallel computing applications?" in *Proceedings of the 39th Annual International Symposium on Computer Architecture*, ser. ISCA, 2012

[22] Shin, J., Hall, M. and Chame, J. "Superword-Level Parallelism in the Presence of Control Flow," in *CGO*. IEEE Computer Society, 2005, pp. 165–175, 2005

[23] Tian, X., Saito, H., Girkar M., Preis, S.V., Kozhukhov, S.S., Cherkasov, A.G., Nelson, C., Panchenko, N., Geva, R. Compiling C/C++ SIMD Extensions for Function and Loop Vectorization on Multicore-SIMD Processors. In proc. of IEEE 26th International Parallel and Distributed Processing Symposium - Multicore and GPU Prog. Models, Lang. and Compilers Workshop, pp.2349 – 2358, 2012

[24] Wu, P., Eichenberger, A. E. and Wang, A., Efficient SIMD Code Generation for Runtime Alignment. *In Proceedings of CGO*, 2005.

Authorized licensed use limited to: Universitaet Linz. Downloaded on March 10,2026 at 12:12:34 UTC from IEEE Xplore. Restrictions apply.